

JobMgr & WeighApp → ROS2 Migration

Transport-only swap (Approach 2) with a full DDS boundary, plus the shared ROS2 Bridge — phased project plan with estimates

Architecture this plan targets (final): JobMgr and WeighApp remain `task::Task` / SConscript-built binaries — no new process, no new class, no new launch mechanism, no re-verification of scheduling/OEL/NVM. Every channel on both apps becomes a DDS-backed shim (not just the ones shared between them), so the two apps are entirely stripped of direct SCS relations. The 3 channels JobMgr and WeighApp share with each other go native DDS-to-DDS, no bridge involved. Every other channel — 13 of JobMgr's 16, plus WeighApp's remaining external channels — routes through one single shared `cpm_scs_bridge` process, which is the only genuinely new binary in the system and the sole SCS→DDS translation seam. This replaces an earlier "pure ROS2 node" design (JobMgrNode/WeighAppNode as standalone processes) that would have required re-verifying real-time scheduling, OEL background tasks, and NVM access — none of that mitigation work is needed under this design, which is why the JobMgr/WeighApp epics below are lighter than earlier drafts.

Estimating assumption: 1 dedicated senior engineer (ROS2 + this AIS codebase), continuous focus, no context-switching to other work, with part-time support from someone who knows the AIS build/deploy pipeline for environment setup and cutover steps. All estimates below are calendar time under that assumption. If 2-3 people can work in parallel, the two app-tracks (JobMgr, WeighApp) can overlap substantially once shared infrastructure (Phase 0-1) is done — see the parallelization note at the end.

~3.5 mo

JOBMGR TRACK

~4.5 mo

WEIGHAPP TRACK

~8 mo

SEQUENTIAL TOTAL

~5.5-6 mo

IF PARALLELIZED

Part I — Shared foundation (build once, both tracks depend on it)

Part II — JobMgr track (5 epics)

Part III — WeighApp track (5 epics)

Part IV — Risk register & parallelization notes

Part I — Shared Foundation

Built once; both JobMgr and WeighApp tracks depend on it. This is the highest-uncertainty phase because it contains the one open technical risk from our research (AIS/ROS2 toolchain ABI compatibility) — don't let either app track start real node-porting work until this is closed out.

ID	Story	Tasks	Est.
EPIC F — Foundation & Shared Infrastructure (~3 weeks)			

F.1	Target platform & toolchain validation	Confirm A7H4's actual GCC/glibc version from a real AIS build environment (<code>gcc --version</code> against the real <code>`tgt_gcc_arm64_guest`</code> toolchain — this is the one thing we couldn't verify from source alone). Confirm it's ABI-compatible with ROS2 Jazzy (GCC 13 / glibc 2.39 / C++17). If there's a gap, scope the toolchain-upgrade work here before anything downstream starts.	3-5d
F.2	ROS2 workspace + SCons cross-linkage	Create the colcon workspace (no YAML generator, per established direction). Package layout is deliberately small: <code>job_mgr_interfaces</code> and <code>weigh_app_interfaces</code> (message/service definitions only — <code>.msg</code> / <code>.srv</code> files need <code>rosidl_generate_interfaces</code> , which only runs inside a colcon-built <code>ament_cmake</code> package) plus <code>cpm_scs_bridge</code> . There is no <code>job_mgr_node</code> / <code>weigh_app_node</code> package — JobMgr and WeighApp stay in the existing <code>SConscript</code> build. This task also wires <code>SConscript</code> 's <code>CPPPATH</code> / <code>LIBPATH</code> to reach the colcon install space (mirroring, in reverse, how the Bridge already reaches into <code>AIS_SDK_ROOT</code>) and adds the new <code>rclcpp</code> / DDS link dependency. Confirm prebuilt <code>ros-jazzy-*</code> apt packages install on A7H4, and decide the domain-ID/discovery config all three DDS participants (JobMgr, WeighApp, Bridge) will share.	2-3d
F.3	Cross-compile / deploy pipeline	Get a build → deploy → run loop working end-to-end: colcon build (interfaces + Bridge) completing before the SCons build (which now depends on colcon's output), deployed and run alongside the existing SCons-built AIS binaries on the same target. This is the "hello world" that proves the two-toolchain sequencing before investing in real ports.	3-5d
F.4	Shim adapter layer	Design and implement <code>RosInputInterface<T></code> / <code>RosOutputInterface<T></code> — header-only template classes matching SCS's <code>InputInterface<T>::get()</code> / <code>OutputInterface<T>::publish()</code> signatures exactly, backed by an <code>rclcpp::Node</code> 's subscriptions/publishers internally. Because both apps are now stripped entirely of direct SCS relations, this shim is used for every channel on both apps, not a subset — worth building once, properly, with unit tests. <code>#include</code> d straight into <code>LpsSaJobMgrApp</code> / <code>LpsSaWeighApp</code> 's existing compilation units; no separate library artifact.	4-5d
F.5	Bridge skeleton	AIS <code>task::Task</code> subclass with an embedded <code>rclcpp</code> node, spun via <code>executor.spin_some()</code> each <code>executive()</code> tick — extending the existing <code>cpm_scs_bridge</code> / <code>WeighScsRosBridge</code> prototype pattern rather than starting from scratch. No real channels wired yet, just the shell proven to run as a registered AIS task.	3-4d

Part II — JobMgr Track

Sequenced first per our earlier analysis: cleanest external boundary (16 channels, all verified), no hardware dependencies, 10 Hz cycle makes bridge latency a non-issue, and 7 of 9 logic files need zero changes.

ID	Story	Tasks	Est.
EPIC J1 — Interface Definition (~2.5 weeks)			
J1.1	Define .msg/.srv for all 16 channels	Field-by-field translation of each SCS struct, field names kept identical to minimize downstream diff (per the convention established during code review). Request/response pairs (Zero, Clear, Reweigh, the 13 UI request types) become <code>.srv</code> ; continuous Tx channels become <code>.msg</code> topics.	4-5d
J1.2	Field-mapping conversion functions	SCS-struct ↔ ROS2-msg translation functions for all 16 channels, unit tested independently of the node itself.	3-4d
J1.3	Review against real structs	Line-by-line check against the actual <code>LpsSaJobMgrTxChannel.h</code> -family headers — this is exactly the discipline that caught the fabricated field names in the earlier pasted document; don't skip it just because it feels redundant.	2d
EPIC J2 — SCS Adapter Replacement (~3 weeks)			
<i>Narrower than a "node port": <code>LpsSaJobMgrApp</code> stays exactly the class/binary/lifecycle it is today. No new class shell, no new timer replacing <code>executive()</code>, no new launch mechanism — the AIS task supervisor starts this exactly as it does now.</i>			
J2.1	Scs.cpp + App.cpp shim swap	All 16 channel objects across <code>LpsSaJobMgrScs.cpp</code> / <code>LpsSaJobMgrApp.cpp</code> change from raw SCS <code>InputInterface<T></code> / <code>OutputInterface<T></code> types to the DDS-backed shim types — call syntax (<code>->get()</code> / <code>->publish()</code>) stays identical, so the surrounding logic in these two files doesn't otherwise change. One new line added inside the existing <code>executive()</code> body: <code>executor_.spin_some()</code> , draining the embedded <code>rclcpp::Node</code> on the same already-correctly-scheduled thread.	5-6d
J2.2	Build/link verification under existing SConscript	Confirm the existing <code>SConscript</code> target still compiles and links cleanly once <code>rclcpp</code> / the DDS implementation are added as dependencies — no ABI or symbol-collision surprises against the AIS toolchain. The 7 untouched logic files (<code>Tasks.cpp</code> , <code>SimpleCal.cpp</code> , <code>Stats.cpp</code> , <code>Cnfg.cpp</code> , <code>Init.cpp</code>) and <code>lps_pass_tracker</code> need no changes at all — this is a build-integrity check, not a port.	2d
J2.3	Direct DDS wiring to WeighApp (3 shared channels)	<code>LpsSaWeighReqstChannelOutput</code> / <code>LpsSaWeighRespChannelInput</code> / <code>LpsSaWeighTxChannelInput</code> talk straight to WeighApp's matching shims over native DDS — no bridge on this leg. Mostly topic/service name and QoS agreement between the two tracks, since the shim plumbing itself was already built in J2.1.	2-3d

J2.4	Component-level testing	<code>LpsSaJobMgrApp</code> in isolation, driven by <code>ros2 topic pub</code> / <code>service call</code> against fake inputs — no bridge, no other apps yet.	5-6d
EPIC J3 — Bridge: JobMgr Legacy Channels (~2.5 weeks)			
<i>13 of JobMgr's 16 channels route through the Bridge (the 3 WeighApp-shared channels went direct DDS in J2.3, never touching the Bridge at all).</i>			
J3.1	5 publisher relays	<code>JobMgrTxChannelOutput</code> , <code>RespChannelOutput</code> , <code>DebugChannelOutput</code> , <code>LoadRecordOutput</code> , <code>OutputChannelOutput</code> — ROS2 → SCS direction.	3-4d
J3.2	8 subscriber relays, sequenced low-risk-first	<code>ReqstChannelInput</code> , <code>SwitchInputScsInput</code> , <code>AisJhm2TxChannelInput</code> , <code>DisplayStateInput</code> first, then the low-frequency ones (<code>ShmClock</code> , <code>DataLink</code> , <code>EventDiagnostic</code> , <code>AutonomyConditionDiagnostics</code>) last.	5-6d
J3.3	Bridge fidelity testing	Loopback validation — publish a known value on the SCS side, confirm it arrives byte-correct on the ROS2 side and vice versa, for all 13 bridged channels.	3-4d
EPIC J4 — Integration, Regression & Cutover (~4.5 weeks)			
J4.1	End-to-end test rig	Real or simulated UI backend (<code>aisJhm2RequestProc</code>), <code>TotalsApp</code> / <code>WorkOrderAssist</code> , <code>aisXcpServer</code> , <code>hornOnStoreTest</code> , <code>CycleSeg</code> , <code>SwitchInputScs</code> , <code>OutputApp</code> — all talking to <code>JobMgrNode</code> through the bridge exactly as they do to the real binary today.	5d
J4.2	Loosely-coupled feature validation	Standby mode, horn-on-store, simple-cal log, tip-off stats — the 4 features with minimal internal coupling, validated first per our earlier risk-ordering.	4-5d
J4.3	Tightly-coupled cluster validation	Pass-tracking, multi-task, store/close — validated together as one integration pass, since that's how they actually execute today.	6-7d
J4.4	Shadow-mode parallel run	Old <code>LpsSaJobMgrApp</code> and new <code>JobMgrNode</code> +bridge running side by side, outputs compared, neither one driving real hardware output yet. Mostly elapsed/monitoring time, not full engineering effort.	7-10d
J4.5	Cutover + stabilization	Swap the AIS task table entry, retire the old binary, monitor. Light-touch monitoring period, not full-time engineering.	2d + 2wk monitor

JobMgr track total: ~14.5 weeks (~3.5 months) sequential, on top of the shared Foundation phase. Down from an earlier draft's ~17 weeks — the reduction comes entirely from J2/J3 no longer carrying a new class shell, a new scheduling model, or 3 extra bridge channels that now go direct.

Part III — WeighApp Track

Deferred until after JobMgr ships, per the explicit earlier decision. Meaningfully larger scope: 48 features vs. 21, a real-time 50 Hz loop (vs. JobMgr's 10 Hz), hardware PWM/CAN dependencies, two build variants (SA/ADV), and — new relative to JobMgr — several direct source dependencies (`lps_weighing` , `lps_common` , `tipoff_assist` , `libsl`) that aren't even populated in the current checkout and would need sourcing before work starts. Also carries a Legal-For-Trade compliance question that JobMgr never had to deal with.

ID	Story	Tasks	Est.
EPIC W0 — Dependency Resolution (~2 weeks, WeighApp-specific, no JobMgr equivalent)			
W0.1	Source the missing submodules	<code>lps_weighing</code> , <code>lps_common</code> , <code>tipoff_assist</code> , <code>libsl</code> are all referenced by WeighApp's <code>SConscript</code> but not populated in this checkout. Locate and pull them before any real analysis of the core weighing computation can start.	3-5d
W0.2	Confirm target-compile cleanly	Verify these compile for A7H4 the same way they do for the legacy A6N2 target — no reason to expect trouble (they're plain C, no AIS/SCS coupling, confirmed for <code>lps_cal</code> already) but not yet verified for the other three.	3-4d
EPIC W1 — Interface Definition (~3.5 weeks, larger than JobMgr's)			
W1.1	Define .msg/.srv for WeighApp's ~25+ channels	Same field-preserving convention as JobMgr. Larger surface: <code>LpsSaWeighTxChannel</code> alone has dozens of fields (dig/dump state, calibration data, LFT/seal state, IMU data).	6-7d
W1.2	Field-mapping conversion functions	Same approach as J1.2, larger scope.	5-6d
W1.3	Review against real structs	Same discipline as J1.3.	3d
EPIC W2 — SCS Adapter Replacement (~4-4.5 weeks)			
<i>Same narrowing as JobMgr's J2: <code>LpsSaWeighApp</code> stays exactly the class/binary/lifecycle it is today, including its existing 50Hz <code>executive()</code> and its OEL-scheduled <code>task_1x()</code> background task (NVM) — neither is touched by this epic.</i>			
W2.1	Scs.cpp + App.cpp shim swap	Larger set of channel objects than JobMgr's — same shim-type-swap technique as J2.1, plus the same single <code>executor_.spin_some()</code> line added inside the existing <code>executive()</code> body.	6-7d
W2.2	Build/link verification + untouched core-pipeline check	Confirm the existing <code>SConscript</code> target still compiles/links cleanly with the new <code>rc1cpp</code> /DDS dependency, and that <code>LpsSaProcess.cpp</code> , the calibration library glue, and the whole dig/dump/best-bucket computation are byte-for-byte unchanged — this is a build-integrity check, not a port, since the class shell never changes.	3-4d
W2.3	Direct DDS wiring to JobMgr (3 shared channels)	WeighApp-side counterpart to J2.3 — the topic/service contract was already agreed there, so this is mostly wiring the WeighApp-side shim instances to match.	1-2d

W2.4	Real-time budget sanity check	Confirm the 20ms cycle budget still holds with <code>spin_some()</code> added to the existing, unmodified <code>executive()</code> — a lighter check than a full re-validation, since the scheduling model itself (OEL, <code>SCHED_RR</code> priority 19, the cycle timer) is untouched and was never the thing being changed.	1-2d
W2.5	Component-level testing	Same approach as J2.4, larger feature surface (48 vs 21).	7-8d
EPIC W3 — Bridge: WeighApp Legacy Channels (~2.5-3 weeks)			
<i>The 3-channel overlap with JobMgr (<code>WeighReqst</code> / <code>WeighResp</code> / <code>WeighTx</code>) never enters the Bridge's scope at all — it's native DDS-to-DDS from W2.3 onward. Only WeighApp's remaining external channels (CycleSeg, aisXcpServer, Cal Manager, etc.) need relays here.</i>			
W3.1	Publisher/subscriber relays	Same pattern as J3.1/J3.2, sized to WeighApp's remaining external channel count.	7-8d
W3.2	Bridge fidelity testing	Same approach as J3.3.	5-6d
EPIC W4 — Integration, Regression, LFT Review & Cutover (~7 weeks)			
W4.1	End-to-end test rig	All AIS-side consumers of WeighApp's Tx (JobMgr, CycleSeg, aisXcpServer) plus the Cal Manager request/response channel.	5-6d
W4.2	Feature validation, staged by risk	Diagnostics/test-fixture features first, then calibration subsystem, then the core weighing pipeline last (highest stakes, most scrutiny).	10-12d
W4.3	Legal-For-Trade compliance review	Not primarily an engineering task — needs input from whoever owns regulatory/ compliance sign-off. Even though the transport-swap approach leaves the actual weighing computation untouched (lower risk than a full rewrite, per our earlier analysis), a formal review is still the responsible move before this ships to a trade-legal device. This is the one item in the whole plan most likely to blow the calendar estimate — it depends on a process outside engineering's control.	2-4wk (parallel, not blocking other work)
W4.4	Shadow-mode parallel run	Same approach as J4.4, longer observation window given the higher stakes.	10-14d
W4.5	Cutover + stabilization	Same shape as J4.5, longer monitoring window.	2d + 3wk monitor

WeighApp track total: ~19 weeks (~4.5 months) sequential. Down from an earlier draft's ~21.5 weeks for the same reason as JobMgr's reduction — no new class shell, no full real-time re-validation, and the 3 JobMgr-shared channels never touch the Bridge in the first place. The LFT review (W4.3) is deliberately shown as running in parallel with other W4 work rather than blocking it, since it's not an engineering dependency — but if it surfaces a real concern, it could add real calendar time on top of this estimate.

Part IV — Risk Register & Parallelization

Risk	Severity	Notes
A7H4 toolchain ABI mismatch with ROS2 Jazzy	High	Not yet confirmed from an actual build environment (F.1). If the AIS SDK's compiler for A7H4 isn't ABI-compatible with GCC 13/glibc 2.39, the Bridge specifically can't link both worlds in one process — this would need toolchain work before anything else proceeds. Close this out first.
Missing WeighApp submodules	High	<code>lps_weighing</code> , <code>lps_common</code> , <code>tipoff_assist</code> , <code>libsl</code> not populated in this checkout (W0). Unknown how hard these are to actually obtain.
LFT compliance review outcome	High	Outside engineering's control (W4.3). Could surface requirements that add scope, not just calendar time.
Shared-library drift (<code>lps_common</code> , <code>lps_app_common</code>)	Medium	These are also used by ListMgrApp, TotalsApp, DemoApp, aisXcpServer — none of which are in scope for this migration. As long as Approach 2 is followed strictly (link the same compiled library, don't reimplement), this stays a non-issue — but it's a discipline to maintain, not a one-time check.
Real-time budget regression on WeighApp's 20ms loop	Low	W2.4 exists to catch this, but the risk itself is smaller than it was under the earlier pure-node design: <code>executive()</code> , its timer, and its OEL/ <code>SCHED_RR</code> scheduling are completely unmodified — the only new cost is one <code>spin_some()</code> call per cycle. JobMgr's 100ms budget has even more slack and was never a concern there.
Shim adapter layer bugs	Low	Mitigated by building it once (F.4), properly, with unit tests, rather than ad-hoc per channel.

Resolved, not tracked below: the earlier open question of how a standalone `JobMgrNode` / `WeighAppNode` process would get launched at boot is moot under this design — both apps keep launching exactly as they do today, via the existing AIS task supervisor and startup scripts. The only new entry in the AIS task table is `cpm_scs_bridge` itself.

Parallelization opportunities, if team size allows

- **W0 (WeighApp dependency resolution) can start immediately after Foundation**, in parallel with the entire JobMgr track — it's just sourcing files, no dependency on JobMgr being done.
- **W1 (WeighApp interface definition) can start once W0 is done**, independent of JobMgr's progress — a second engineer could be running the WeighApp interface-definition epic while the first is still in JobMgr's node-port phase.
- **The LFT review (W4.3) should be kicked off as early as the interfaces are stable** (end of W1), not held until integration testing — it's a long-lead item with no engineering dependency, so starting it early is pure upside.
- What genuinely can't parallelize: **Bridge work for either app depends on that app's interface definition being done first**, and **cutover for either app should not happen before its own shadow-mode run completes**.

With 2-3 people exploiting the parallelization above, combined calendar time could realistically compress from ~8 months sequential to **~5.5-6 months**. With 1 person, treat the ~8 month sequential estimate as the honest number.

Source basis: all scope figures (16/48 channels, 21/48 features, 7/9 unchanged JobMgr files, 4 missing WeighApp submodules) are grounded in the verified analysis earlier in this engagement — see WeighApp-JobMgr-Code-Walkthrough.pdf and the flow-diagram artifact for the underlying evidence.